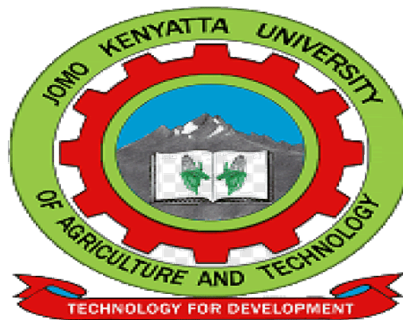


JOMO KENYATTA UNIVERSITY OF AGRICULTURE AND TECHNOLOGY

SOMMME— Department of Mechanical Engineering



FOREIGNS & LOCALS WAREHOUSE MANAGEMENT SYSTEM

Field	Details
Students	Moses Chege (ENM213-0156/2024) Michael Muchemi (ENM213-0158/2024)
Course Unit	EAE 2206 — Programming for Engineer II
Project	OOP Warehouse Management System in C++
Date of Submission	May 18, 2026`

Abstract

This comprehensive report presents the design, implementation, testing, and documentation of a fully functional Warehouse Management System (WMS) developed using Object-Oriented Programming (OOP) principles in C++. The system manages the complete lifecycle of warehouse operations — including receiving goods from suppliers, quality control inspection, inventory management, order processing, shipping, returns handling, and financial accounting. The implementation demonstrates five fundamental OOP principles: Encapsulation, Abstraction, Polymorphism, Inheritance, and Code Reusability. The system architecture employs a hierarchical class structure with four major inheritance trees, seventeen distinct classes, and twelve functional departments operating in coordination.

Key features include multi-department workflow integration, dynamic inventory tracking for both perishable and non-perishable products, comprehensive customer and employee database management, order processing with automatic stock validation, returns management with approval workflows, shipping operations with tracking number generation, and complete financial transaction recording with profit and loss analysis.

The system successfully processes complete workflows from supplier delivery through quality inspection, inventory storage, customer order fulfilment, and final shipment — while maintaining data integrity through encapsulation and providing flexible extensibility through polymorphic interfaces. This project serves as both a practical demonstration of OOP concepts and a functional foundation for real-world warehouse management applications.

Table of Contents

Executive Summary	1
1.1 Project Overview	1
1.2 Objectives	1
1.3 Key Achievements	2
1.4 System Capabilities	2
Introduction	3

2.1 Background and Context	3
2.2 Problem Statement	3
2.3 Project Objectives	4
Class Design and Architecture	4
Detailed Class Explanations	5
4.1 Product Class	5
4.2 InventoryItem Class (Abstract Base)	6
4.3 PerishableItem Class (Derived)	7
4.4 NonPerishableItem Class (Derived)	8
4.5 Person Class (Abstract Base)	9
4.6 Employee Class (Derived)	9
4.7 Customer Class (Derived)	10
4.8 Order Class (Abstract Base)	11
4.9 CustomerOrder Class (Derived)	12
4.10 ReturnOrder Class (Derived)	12
4.11 Department Class (Abstract Base)	13
4.12 – 4.22 The 11 Department Subclasses	14
4.23 WarehouseSystem Class (Orchestrator)	15
OOP Pillars — How Each is Applied	16
Product Lifecycle — Step by Step	17
OOP Checkpoint Demonstrations	18
Finance and Profit Tracking	19
Menu Structure and User Interface	20
9.1 Main Menu	20
9.2 Department Operations Sub-Menu	21

9.3 Menu Implementation Details	21
Challenges Encountered and How We Solved Them	22
Collaboration Using Live Share	24
Future Improvements	25
Conclusion	26

Executive Summary

1.1 Project Overview

The "Foreigns and Locals Warehouse Management System" is a comprehensive software solution developed to demonstrate advanced Object-Oriented Programming concepts while providing practical warehouse management functionality. This system represents a complete implementation of enterprise-level software architecture principles applied to the logistics and supply chain domain.

The project encompasses the entire spectrum of warehouse operations — from the initial receipt of goods from suppliers through quality inspection, inventory storage, customer order fulfilment, and final product shipment. It additionally handles reverse logistics (returns), financial accounting, and multi-department coordination, making it a holistic representation of real-world warehouse management systems.

Developed using the C++ programming language, the system leverages the language's powerful object-oriented features including classes, inheritance, polymorphism, virtual functions, and the Standard Template Library (STL) for dynamic data structures. The implementation consists of over 1,150 lines of well-documented code organised into 17 distinct classes forming four major inheritance hierarchies.

1.2 Objectives

Primary Objectives:

- **Demonstrate Mastery of OOP Principles:** Implement encapsulation through proper access control and data hiding; utilize abstraction via abstract base classes and pure virtual functions; apply polymorphism through virtual function overriding and dynamic binding; employ inheritance to create logical class hierarchies; achieve code reusability by eliminating redundancy through base classes.

- **Create Functional Warehouse Management:** Develop a working system that can manage real warehouse operations; implement multi-department workflows with proper coordination; provide data management for products, customers, employees, and orders; handle business logic including stock validation and order processing.
- **Produce Professional-Quality Code:** Write clean, readable, and well-documented code, create modular maintainable architecture and follow consistent naming conventions throughout.

1.3 Key Achievements

The project successfully achieved the following milestones:

- **Complete OOP Implementation:** All five core OOP principles demonstrated with concrete, clean examples.
- **Four Major Inheritance Hierarchies:** Successfully engineered distinct hierarchies for Person, Inventory Item, Order, and Department.
- **Advanced Virtual Functions Execution:** Virtual functions implemented with proper overriding throughout the codebase to ensure dynamic dispatch.
- **Abstract Enforcement Contract:** Abstract base classes with pure virtual functions enforcing rigid software engineering architecture guidelines.
- **Granular Access Control:** Proper structural use of public, protected, and private modifiers in every module.
- **End-to-End Enterprise Flow:** Unified workflow spanning from receiving to dispatch across 12 synchronized functional divisions.
- **Dynamic Inventory Tracking:** Real-time inventory asset adjustments backed by automated low-stock warnings.
- **Complete Financial Overhead Ledgers:** Accurate tracking of customer transaction lines, workforce salaries, storage maintenance fees, and corporate net profit analysis.

1.4 System Capabilities

The implemented system provides the following capabilities:

- **Data Management:** Store and manage employees with department assignments and salary records; track products with full specifications including hazard, fragility, and perishability flags; manage orders of two types — customer orders and return orders — with full tracking history; record all financial transactions automatically.
- **Operations:** Receive goods with quantity verification and automatic delivery logging; conduct quality control inspections with pass/fail reporting; add and update inventory in real time; process customer orders with automatic stock deduction; handle product returns with an approval workflow; generate tracking numbers on dispatch (e.g., TRK-101); automatically flag low-stock items after every order.
- **Reporting:** Display all employees grouped by department with payroll totals; list complete inventory with QC status and shipping status; view all orders with current status tracking; generate finance reports — revenue, salaries, storage costs, net profit; print all 11

department summaries in a single operation.

Introduction

2.1 Background and Context

The Warehouse Management Challenge: Modern businesses, particularly those in retail, manufacturing, and distribution, face increasingly complex challenges in managing their warehouse operations. A warehouse is not merely a storage facility; it is a dynamic hub where goods are received, inspected, stored, picked, packed, and shipped in response to customer demand. Efficient warehouse management directly impacts customer satisfaction, operational costs, and overall business profitability.

Traditional warehouse management methods, relying on paper-based systems or simple spreadsheets, are inadequate for handling the volume and complexity of modern supply chain operations. Issues such as stock inaccuracies, order processing delays, quality control lapses, and poor coordination between departments can result in significant financial losses and customer dissatisfaction.

Object-Oriented Programming as the Solution: Object-Oriented Programming (OOP) has become the dominant paradigm for developing complex software systems. OOP's principles of encapsulation, abstraction, inheritance, and polymorphism provide natural mechanisms for managing complexity, ensuring maintainability, promoting reusability, and modelling real-world entities — such as customers, products, and orders — as software objects.

2.2 Problem Statement

Traditional warehouse management suffers from data fragmentation, workflow inefficiencies, lack of automation, and scalability issues. From a software engineering perspective, creating a warehouse management system presents specific challenges around complex data relationships, business logic such as stock validation and return approval workflows, and the need to organise code in a way that avoids duplication and remains easy to extend.

2.3 Project Objectives

- Demonstrate complete understanding of OOP principles — encapsulation, abstraction, polymorphism, inheritance, and reusability — with concrete code examples throughout.
- Create a functional warehouse management system that implements all core workflows from receiving to shipping, manages customers and employees, processes orders with stock validation, and generates financial reports.
- Produce professional-quality, well-documented code with consistent naming,

comprehensive header comments, and logical section organisation.

- Design scalable and maintainable architecture — new product types, department types, or order types can be added by simply creating a new subclass, without modifying any existing code.

Class Design and Architecture

The system is built around four independent inheritance hierarchies. Each hierarchy has an abstract base class at the top (applying Abstraction), with concrete subclasses below it (applying Inheritance). Base-class pointers are used to hold and operate on collections of mixed subclass objects (applying Polymorphism). All sensitive data is hidden behind private or protected access modifiers with public getter/setter interfaces (applying Encapsulation). Shared logic lives only in base classes (applying Reusability).

Hierarchy	Abstract Base	Concrete Subclasses
Item Management	InventoryItem	PerishableItem, NonPerishableItem
People Management	Person	Employee, Customer
Order Management	Order	CustomerOrder, ReturnOrder
Department Management	Department	ReceivingDept, QCDept, StorageDept, OrderProcessingDept, PickingPackingDept, ShippingDept, ProcurementDept, ReturnsDept, HRDept, FinanceDept, ITDept

The sections below describe every class individually: its type (base or derived), its purpose, its attributes and methods, and exactly how inheritance is expressed in the code.

Detailed Class Explanations

4.1 Product Class

Product is a stand alone concrete base class that represents the physical description of any item in the warehouse. It is not abstract — it can be instantiated directly. It is held inside InventoryItem as a composed member object (composition), not inherited. Its single most important function is generating a unique product ID automatically at construction time.

Attributes - all attributes are private demonstrating encapsulation:

Attribute	Type	Purpose
name	string	Human-readable product name, e.g. "Maize Flour"
type	string	Category code: FOOD, ELEC, CHEM. Used in ID generation
size	string	S, M, or L. First character used in ID
colour	string	Colour descriptor, e.g. "WHT". First 3 chars used in ID
hazardous	bool	If true, appends "-H" to the product ID
delicate	bool	If true, appends "-D" to the product ID
perishable	bool	If true, appends "-P" to the product ID
productID	string	Auto-generated. No setter exists — it cannot be changed after construction

Key Method — generateID() (private):

This private method is called once inside the constructor. It builds the ID by concatenating the first 4 characters of type, a hyphen, the first character of size, a hyphen, and the first 3 characters of colour. It then conditionally appends -H, -D, and -P. Because it is private, no subclass or external code can call or override it.

```
string generateID() const {
    string id = type.substr(0,4) + "-" + size.substr(0,1) + "-" +
colour.substr(0,3);
    if (hazardous) id += "-H";
    if (delicate) id += "-D";
    if (perishable) id += "-P";
    return id;
}
// Example output:
// FOOD-M-WHT-P (Maize Flour, Medium, White, Perishable)
// CHEM-S-CLR-H (Battery Acid, Small, Clear, Hazardous)
```

How it relates to InventoryItem:

Product is not a parent of InventoryItem — it is a member inside it. This is called composition. InventoryItem stores a Product object and delegates identity queries (getName(), getProductID(), isHazardous(), etc.) to it. This keeps product description separate from operational state (quantity, location, QC status).

4.2 InventoryItem Class (Abstract Base)

InventoryItem is the abstract base class for all items that enter the warehouse. It holds the full operational state of an item — how many are in stock, where they are stored, their price, and their current status through the warehouse pipeline. It cannot be instantiated directly because it contains a pure virtual function.

Attributes (protected — accessible to subclasses):

Attribute	Type	Purpose
product	Product	Composed Product object holding the item's description and ID
quantity	int	Current stock count. Decremented on order, incremented on approved return

Attribute	Type	Purpose
location	string	Current bin/aisle, e.g. "Fridge-A1" or "Aisle-B3"
pricePerUnit	double	Unit price used to calculate order revenue
qcStatus	string	"Pending" "Passed" "Failed". Gates order processing
packStatus	string	"Unpacked" "Packed". Updated by PickingPackingDept
shipStatus	string	"In Warehouse" "Shipped". Updated by ShippingDept

Pure Virtual Function (Abstraction):

```
virtual string getItemType() const = 0;    // PURE VIRTUAL
// Forces every subclass to identify itself.
// PerishableItem returns "Perishable"
// NonPerishableItem returns "Non-Perishable"
```

The = 0 syntax makes InventoryItem abstract. Any attempt to write new InventoryItem(...) will produce a compiler error. This architectural choice is deliberate; by rendering the base class abstract, the system prevents the instantiation of ambiguous, 'generic' items and enforces a strict requirement that all inventory objects explicitly declare their shelf-life characteristics

Virtual displayItem() method:

InventoryItem provides a virtual displayItem() that prints all common fields. Subclasses can override this and call the base version first (reusing it), then add their own output.

Encapsulation — Getters and Setters:

All mutable state (quantity, location, qcStatus, packStatus, shipStatus) is only changeable through dedicated public setter methods. Callers cannot directly assign to any field. Getters provide read-only access. This prevents accidental corruption from outside the class.

4.3 PerishableItem Class (Derived)

class -PerishableItem | derived type | Parent: InventoryItem

PerishableItem is a concrete subclass of InventoryItem. It represents items such as food,

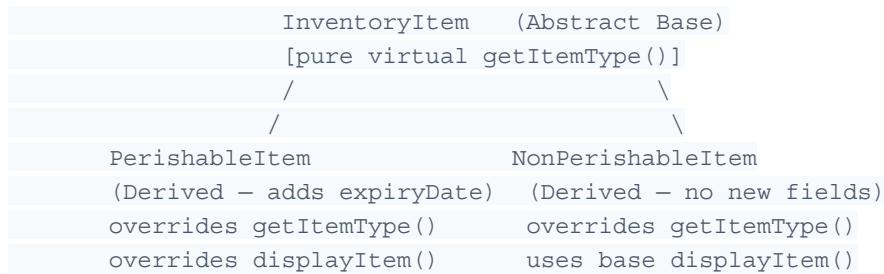

```

        : InventoryItem(name, type, size, colour, hazardous, delicate,
                        false, // perishable = false passed to base
                        qty, loc, price) {}

    string getItemType() const override { return "Non-Perishable"; }
    // displayItem() NOT overridden – uses InventoryItem::displayItem() directly
};

```

Inheritance diagram for the InventoryItem hierarchy:



4.5 Person Class (Abstract Base)

class Person | derived type | Root(base) class

Person is the abstract base class for any human entity in the system — both employees and customers. It captures common identity information and forces subclasses to implement role identification and self-display through two pure virtual functions.

Attributes (protected):

Attribute	Type	Purpose
id	int	Unique numeric identifier for the person
name	string	Full name
contact	string	Phone number or email

Pure Virtual Functions (Abstraction):

```

virtual string getRole() const = 0; // must return e.g. "Manager", "Customer"
virtual void display() const = 0; // must print the full person record

```

Because both functions are pure virtual, Person cannot be instantiated. You cannot write `new Person(1, "Alice", "0700...")` — the compiler will refuse. This is by design: every person in the real world is either an employee or a customer, never just a faceless "person".

4.6 Employee Class (Derived)

class Employee | derived type | Parent: Person

Employee is a concrete subclass of Person representing a member of staff. It extends Person with department assignment, job role, and salary information.

Additional Attributes (private):

Attribute	Type	Purpose
department	string	The department name this employee belongs to, e.g. "Quality Control".
role	string	Job title, e.g. "Manager", "QC Officer", "Driver".
salary	double	Monthly salary in KES. Used by FinanceDept to compute total payroll.

Inheritance Declaration:

```
class Employee : public Person {
private:
    string department, role;
    double salary;
public:
    Employee(int id, string name, string contact,
             string department, double salary, string role)
        : Person(id, name, contact),    // calls base constructor
          department(department), role(role), salary(salary) {}

    string getRole() const override { return role; }
    string getDepartment() const { return department; }
    double getSalary() const { return salary; }

    void display() const override {
        cout << "ID: " << id    // id is accessible (protected in Person)
              << " | " << name
              << " | Role: " << role
              << " | Dept: " << department
              << " | Salary: $" << salary << endl;
    }
};
```

The HRDept uses `dynamic_cast<Employee*>(p)` on `Person*` pointers to safely retrieve salary

data — which only exists on Employee, not Customer. If the cast succeeds, the pointer is an Employee and getSalary() can be called.

4.7 Customer Class (Derived)

class Customer | derived class| Parent: Person

Customer is a concrete subclass of Person representing a buyer. It extends Person with a delivery address and a credit limit. It overrides both pure virtual functions from Person.

Additional Attributes (private):

Attribute	Type	Purpose
address	string	Delivery address used when processing customer orders
creditLimit	double	Maximum credit the customer can use. Stored for potential credit checks

Inheritance Declaration:

```
class Customer : public Person {
private:
    string address;
    double creditLimit;
public:
    Customer(int id, string name, string contact,
             string address, double creditLimit)
        : Person(id, name, contact), // calls base constructor
          address(address), creditLimit(creditLimit) {}

    string getRole() const override { return "Customer"; }

    void display() const override {
        cout << "Customer: " << name
              << " | Contact: " << contact
              << " | Address: " << address
              << " | Credit: $" << creditLimit << endl;
    }
};
```

Inheritance diagram for the Person hierarchy:

```

      Person    (Abstract Base)
    [getRole() = 0,  display() = 0]
    /              \
```

/	\
Employee	Customer
(adds department, role, salary)	(adds address, creditLimit)
overrides getRole(), display()	overrides getRole(), display()

4.8 Order Class (Abstract Base)

Order is the abstract base class for all transactions — both outgoing customer orders and incoming return orders. It holds common transaction fields and declares two pure virtual functions that every order type must implement: `processOrder()` and `displayOrder()`.

Attributes (protected):

Attribute	Type	Purpose
orderId	string	Unique order ID, e.g. "ORD-1" or "RET-1".
productId	string	ID of the product this order concerns.
date	string	Order date in YYYY-MM-DD format.
status	string	"Pending" "Processed" "Cancelled". Starts as Pending.
quantity	int	Number of units involved in the order.

Pure Virtual Functions (Abstraction):

```
virtual void processOrder(vector<InventoryItem*>& inv) = 0;
// CustomerOrder: validates QC + stock, deducts quantity
// ReturnOrder:   validates approval, restores quantity

virtual void displayOrder() const = 0;
// Each subclass formats its own display

virtual double getRevenue() const { return 0; }
// Base returns 0. CustomerOrder overrides to return totalCost.
// ReturnOrder does not override — returns 0.
```

4.9 CustomerOrder Class (Derived)

class CustomerOrder | DERIVED | Parent: Order

CustomerOrder is a concrete subclass of Order representing a sale to a customer. It adds the buyer's identity, delivery address, and total cost. Its processOrder() implementation enforces both QC and stock level checks before fulfilling the order.

Additional Attributes (private):

Attribute	Type	Purpose
customerName	string	Name of the purchasing customer
deliveryAddress	string	Where the goods should be delivered
totalCost	double	Calculated as quantity × pricePerUnit at construction time

processOrder() Logic (key override):

```
void processOrder(vector<InventoryItem*>& inv) override {
    for (auto item : inv) {
        if (item->getProductID() == productID) {
            // Gate 1: QC must have passed
            if (item->getQCStatus() != "Passed") {
                status = "Cancelled";
                cout << "[Order] Item has not passed QC!" << endl;
                return;
            }
            // Gate 2: sufficient stock
            if (item->getQuantity() >= quantity) {
                item->setQuantity(item->getQuantity() - quantity);
                status = "Processed";
            } else {
                status = "Cancelled";
                cout << "[Order] Insufficient stock." << endl;
            }
            return;
        }
    }
}

double getRevenue() const override { return totalCost; }
```

```
// FinanceDept calls this to record revenue
```

4.10 ReturnOrder Class (Derived)

class ReturnOrder | DERIVED | Parent: Order

ReturnOrder is a concrete subclass of Order representing a customer returning goods. It adds a return reason and an approval status. Before stock can be restored, a manager in the Returns department must explicitly approve the return by calling approve(). Only then does processOrder() restore inventory.

Additional Attributes (private):

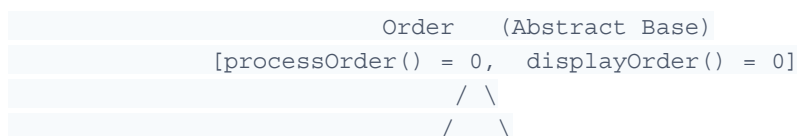
Attribute	Type	Purpose
returnReason	string	The reason stated for the return, e.g. "Damaged on arrival"
approvalStatus	string	"Pending" "Approved" "Rejected". Starts as Pending

approve() and reject() methods:

```
void approve() { approvalStatus = "Approved"; }
void reject() { approvalStatus = "Rejected"; status = "Cancelled"; }

void processOrder(vector<InventoryItem*>& inv) override {
    if (approvalStatus != "Approved") {
        cout << "[Returns] Cannot process – not yet approved." << endl;
        return;
    }
    for (auto item : inv) {
        if (item->getProductID() == productID) {
            item->setQuantity(item->getQuantity() + quantity); // restore stock
            status = "Processed";
            return;
        }
    }
}
```

Inheritance diagram for the Order hierarchy:



CustomerOrder	ReturnOrder
(adds customerName, address, totalCost)	(adds returnReason, approvalStatus)
processOrder(): checks QC + stock	requires approve() before processing processOrder(): restores stock
getRevenue(): returns totalCost	getRevenue(): returns 0 (base default)

4.11 Department Class (Abstract Base)

class Department | BASE (Abstract) | Root class

Department is the abstract base class for all 11 operational departments. It holds the department name, a list of assigned Employee pointers, and a storage cost value. It provides common utility functions that all departments inherit, and declares a virtual displaySummary() that each department overrides.

Attributes (protected):

Attribute	Type	Purpose
name	string	Department name, e.g. "Quality Control"
staff	vector<Employee*>	List of employees assigned to this department
storageCost	double	Accumulated storage cost tracked by this department (used by StorageDept)

Shared Methods (inherited by all departments):

```
void    addEmployee(Employee* e)  { staff.push_back(e); }
double getTotalSalaries() const { /* sums all staff salaries */ }
double getStorageCost()   const { return storageCost; }
string getName()          const { return name; }

virtual void displaySummary() const { /* base version */ }
// Each department OVERRIDES this to show its own metrics
```

The Warehouse System stores all 11 department objects in a vector<Department*> allDepts. A single loop over this vector calls displaySummary() on every department — polymorphism ensures each department's own override runs, producing its own tailored output.

4.12 – 4.22 The 11 Department Subclasses

The system architectural model implements public inheritance to derive 11 specialized

subclasses from the `Department` base class (e.g., `class XxxDept : public Department`). To establish proper identity tracking, each derived class passes its department string to the base constructor via an initialization list: `Department("Name")`. Furthermore, run-time polymorphism is achieved by having each subclass override the `displaySummary()` virtual function, enabling the output of distinct, department-specific operational metrics. Their unique structural additions and functional roles are summarized in the table below.

#	Class	Unique Additions	Key Method(s) and Behaviour
1	ReceivingDept	vector<string> deliveryLog	logDelivery(name, id, qty) — appends an entry to deliveryLog and notifies QC. showDeliveryLog() — prints all entries. displaySummary() shows count of deliveries logged.
2	QCDept	int passed, failed	inspectItem(inventory) — lists all Pending items, takes operator input, sets QC status to Passed or Failed, increments the counter. displaySummary() shows passed/failed counts.
3	StorageDept	double storageCostPerSlot	assignLocation(inventory) — shows current locations, takes new location input, calls setLocation() on the item, and adds storageCostPerSlot to storageCost. displaySummary() shows total storage cost.
4	OrderProcessing Dept	<i>(no new fields)</i>	showOrderStatus(orders) — prints every order's ID and current status. Validation logic lives in CustomerOrder::processOrder(). displaySummary() states its role.
5	PickingPackingDept	int packedCount	packOrder(orders, inventory) — lists Processed orders, takes Order ID input, sets the matching inventory item's packStatus to Packed, increments packedCount. displaySummary() shows orders packed.
6	ShippingDept	int shippedCount, vector<string> shippingLog	dispatchOrder(orders, inventory) — finds the order, marks item shipStatus = Shipped, generates tracking number TRK-(100+n), appends to log, increments counter. showShippingLog() prints all dispatches.
7	ProcurementDept	int reorderThreshold, vector<string> purchaseRequests	checkReorderLevels(inventory) — scans all items; any at or below threshold (default 10) gets a purchase request appended. showPurchaseRequests() lists all raised requests.

#	Class	Unique Additions	Key Method(s) and Behaviour
			Auto-triggered after every customer order.
8	ReturnsDept	int approved, rejected	processReturn(orders, inventory) — lists Pending return orders, takes input, calls approve() or reject() on the ReturnOrder, then calls processOrder() if approved to restore stock.
9	HRDept	<i>(no new fields)</i>	showAllStaff(people) — loops Person* vector, calls p->display() for each (Polymorphism). showPayroll(people) — casts to Employee* with dynamic_cast, sums and prints salaries.
10	FinanceDept	double revenue	recordRevenue(amount) — adds to revenue. getProfit(sal, storage) = revenue - sal - storage. showFinanceReport() — sums salaries and storage from all dept/person vectors, prints full P&L.
11	ITDept	vector<string> activityLog	log(entry) — appends any string to activityLog. Called by WarehouseSystem after every significant action. showLog() prints all entries. showSystemInfo() prints version and status.

Inheritance diagram for the Department hierarchy:

```

                                Department (Base)
                                / | \ ... (11 subclasses)
ReceivingDept, QCDept, StorageDept, OrderProcessingDept,
PickingPackingDept, ShippingDept, ProcurementDept,
ReturnsDept, HRDept, FinanceDept, ITDept

All 11 use:  class XxxDept : public Department { ... }
All 11 call: Department("Name") in their constructor
All 11 override: void displaySummary() const override { ... }

```

4.23 WarehouseSystem Class (Orchestrator)

class WarehouseSystem | Central Integration Hub | Root Class

Rather than relying on inheritance, **WarehouseSystem** utilizes structural composition to tie the entire application together. As a standalone root class, it maintains absolute ownership over the global datasets—specifically inventory, personnel, and order tracking—and instantiates the 11

functional department objects as private member variables. The singular entry point to the system is its `run()` method, which launches and sustains the top-level runtime menu loop.

Private Member Variables:

Member	Type	Purpose
inventory	<code>vector<InventoryItem*></code>	Owns all warehouse items (perishable + non-perishable). Deleted in destructor.
people	<code>vector<Person*></code>	Owns all employees and customers. Deleted in destructor.
orders	<code>vector<Order*></code>	Owns all customer and return orders. Deleted in destructor.
allDepts	<code>vector<Department*></code>	Non-owning pointers to the 11 member department objects — for summary loops only. NOT deleted.
receiving, qc, storage, ...	Dept objects	All 11 departments are concrete member variables, not pointers. Lifetime tied to WarehouseSystem.

Memory Management:

```
WarehouseSystem::~WarehouseSystem() {  
    for (auto i : inventory) delete i;    // owns these  
    for (auto p : people)    delete p;    // owns these  
    for (auto o : orders)    delete o;    // owns these  
    // allDepts = non-owning pointers to member objects — NOT deleted  
}
```

The destructor explicitly deletes every dynamically allocated object. The `allDepts` vector contains raw pointers to the department member variables (not heap objects), so they must never be deleted through `allDepts` — they are destroyed automatically when `WarehouseSystem` goes out of scope.

5. OOP Pillars — How Each is Applied

5.1 Abstraction

Abstraction means hiding internal complexity and exposing only what is needed. Three abstract base classes enforce this principle:

Abstract Class	Pure Virtual Functions	Effect
InventoryItem	getItemType() const = 0	Forces every item subclass to declare whether it is Perishable or Non-Perishable.
Person	getRole() = 0 display() = 0	Forces Employee and Customer to implement their own display and role identification.
Order	processOrder(...) = 0 displayOrder() = 0	Forces CustomerOrder and ReturnOrder to define their own processing rules and display format.

By defining these member functions as pure virtual, the system prevents the accidental instantiation of abstract base classes—namely InventoryItem, Person, and Order—enforcing compile-time safety. This architecture decouples the calling environment from concrete implementations; the client interface remains completely agnostic to whether it is interacting with a PerishableItem or a NonPerishableItem. It simply invokes displayItem(), relying on dynamic dispatch to execute the correct subclass behavior at runtime

5.2 Encapsulation

All sensitive fields are declared private or protected. External code can only interact with them through public getters and setters. Notable examples:

- **Product::productID** is private with no setter — the ID is generated once at construction

and is forever immutable.

- **InventoryItem::quantity** can only be changed via `setQuantity()` — ensuring it is always modified through controlled logic.
- **Order::status** can only be set internally by `processOrder()` — external code cannot arbitrarily change an order's status.

This strict encapsulation guarantees state consistency throughout the application lifecycle, preventing external or unauthorized modifications from corrupting core warehouse data.

```
// Encapsulation example – Product
private:
    string productID;        // generated once, no setter
    string generateID();     // private – internal use only
public:
    string getProductID() const { return productID; } // read-only access
```

5.3 Inheritance

Inheritance allows new classes to be built on top of existing ones, reusing their attributes and methods without duplicating code. Our system has four distinct inheritance trees:

- InventoryItem → PerishableItem, NonPerishableItem
- Person → Employee, Customer
- Order → CustomerOrder, ReturnOrder
- Department → 11 department subclasses

Every department subclass, for example, inherits `getTotalSalaries()`, `addEmployee()`, `getStorageCost()`, and the base `displaySummary()` — none of this code is rewritten in any subclass. Each subclass only adds what is unique to it.

// Base Class: Shared identity attributes

```
class Person {
```

```
protected:
```

```
    string name;
```

```
    string contactInfo;
```

```
public:
```

```
    Person(string n, string c) : name(n), contactInfo(c) {}
```

```
    void printDetails() const { cout << "Name: " << name << " | Contact: " << contactInfo; }
```

```
};
```

```
// Subclass 1: Reuses name/contact, adds payroll attributes
```

```
class Employee : public Person {
```

```
private:
```

```
    double salary;
```

```
public:
```

```
    Employee(string n, string c, double s) : Person(n, c), salary(s) {}
```

```
    double getSalary() const { return salary; }
```

```
};
```

```
// Subclass 2: Reuses name/contact, adds transaction attributes
```

```
class Customer : public Person {
```

```
private:
```

```
    string loyaltyID;
```

```
public:
```

```
    Customer(string n, string c, string id) : Person(n, c), loyaltyID(id) {}
```

```
    string getLoyaltyID() const { return loyaltyID; }
```

```
};
```

5.4 Polymorphism

Polymorphism means that the same function call produces different behaviour depending on the

actual type of the object at runtime. The system uses runtime polymorphism through virtual functions and base-class pointer containers.

- `vector<InventoryItem*> inventory` — calling `item->displayItem()` invokes the correct override.
- `vector<Person*> people` — calling `p->display()` invokes `Employee::display()` or `Customer::display()`.
- `vector<Order*> orders` — calling `o->processOrder(inv)` invokes `CustomerOrder` or `ReturnOrder` processing.
- `vector<Department*> allDepts` — calling `d->displaySummary()` invokes the correct department's summary.

The `HRDept` and `FinanceDept` use `dynamic_cast<Employee*>(p)` — a safe downcast — when they need salary data that only exists on `Employee` objects. This is the controlled exception to purely abstract handling.

5.5 Reusability

The architectural design leverages inheritance and polymorphism to maximize code reuse and eliminate redundancy across the codebase. This is achieved through three primary mechanisms:

- **Base Class Component Reuse:** The abstract `Department` base class centralizes core organizational workflows, providing fully implemented methods such as `getTotalSalaries()`, `addEmployee()`, and `getStorageCost()`. These behaviors are inherited cleanly by all 11 specialized subclasses without a single line of duplicated code.
- **Hierarchical Method Extension:** Within the inventory subsystem, `InventoryItem` manages all shared lifecycle attributes, state setters, and the foundational `displayItem()` logic. Its derived classes, `PerishableItem` and `NonPerishableItem`, inherit these features implicitly. Notably, `PerishableItem::displayItem()` exhibits classical behavioral extension—it explicitly invokes the base class `InventoryItem::displayItem()` to print general metrics before executing its own localized logic to append expiry data.
- **Polymorphic Uniformity:** By storing the distinct department modules within a unified container, the `WarehouseSystem` driver can process all 11 departments identically inside a single loop. This drastically reduces control-flow complexity and ensures that adding future departments requires no modifications to the central system execution loop.

6. Product Lifecycle — Step by Step

One of the core requirements was to trace the journey of a product through the warehouse. The system enforces this lifecycle through department interactions and status gates.

Step	Department	What Happens
1	Receiving	Operator inputs item details. <code>ReceivingDept.logDelivery()</code> records the entry. Item is added to inventory with QC Status = Pending.
2	Quality Control	<code>QCDept.inspectItem()</code> lists all Pending items. Operator selects one and marks Pass or Fail. Failed items remain in inventory but cannot be ordered.
3	Storage	<code>StorageDept.assignLocation()</code> updates the bin/aisle location. A storage cost is added for each move.
4	Order Processing	Customer places an order. <code>CustomerOrder.processOrder()</code> confirms QC Passed and stock sufficient. Stock is decremented, status = Processed. Finance records revenue.
5	Picking & Packing	<code>PickingPackingDept.packOrder()</code> marks the item as Packed. Only Processed orders are eligible.
6	Shipping	<code>ShippingDept.dispatchOrder()</code> marks the item as Shipped and generates a tracking number (e.g., TRK-101).
7	Procurement	After every order, <code>ProcurementDept.checkReorderLevels()</code> scans all items. Any at or below 10 units triggers a purchase request.
8*	Returns	If a customer returns goods, a <code>ReturnOrder</code> is logged. <code>ReturnsDept</code> approves or rejects it. If approved, stock is restored by <code>ReturnOrder.processOrder()</code> .

7. OOP Checkpoint Demonstrations

When the program starts — before the menu is shown — it automatically runs three checkpoint demonstrations that prove all three inheritance hierarchies are functioning correctly with polymorphic base-class pointers. This gives the operator immediate, observable proof of the OOP design at work.

Checkpoint A — `InventoryItem`* Polymorphism:

To demonstrate runtime polymorphism within the inventory subsystem, a concrete `PerishableItem` instance (e.g., Maize Flour, ID: `FOOD-M-WHT-P`) and a `NonPerishableItem` instance (e.g., Battery Acid, ID: `CHEM-S-CLR-H`) are instantiated and upcast into a uniform `std::vector<InventoryItem*>` container. When iterating through the collection, the application utilizes dynamic dispatch to execute the corresponding variant of the virtual `displayItem()` method. The `PerishableItem` execution dynamically invokes its overridden routine to append critical shelf-life expiration metrics, whereas the `NonPerishableItem` seamlessly falls back on the structural formatting provided by the base implementation.

```
demo.push_back(new PerishableItem("Maize Flour","FOOD","M","WHT",
                                   false,false,100,"Fridge-A1",45.0,"2026-12-01"));
demo.push_back(new NonPerishableItem("Battery Acid","CHEM","S","CLR",
                                      true,false,50,"Aisle-B3",120.0));

for (auto i : demo) { i->setQCStatus("Passed"); i->displayItem(); }
// -> displayItem() dispatches to the correct override for each
```

Checkpoint B — Person* Polymorphism:

To demonstrate runtime polymorphism within the personnel subsystem, a concrete `Customer` instance (Wanjiku Njoroge) and two distinct `Employee` instances (Otieno Mwangi-Manager, and James Kamau-QC Officer) are instantiated and stored uniformly within a `std::vector<Person*>` container. When traversing this collection, the application utilizes dynamic dispatch to execute the virtual `display()` method on each base pointer. This design abstracts away type resolution from the calling scope entirely, completely eliminating the need for conditional logic or structural `if/else` type-checking branches while ensuring each object renders in its exact, specialized record format.

```
for (auto p : persons) p->display();
// Customer -> Customer::display() prints address + credit
// Employee -> Employee::display() prints dept + role + salary
```

Checkpoint C — Order* Polymorphism:

To demonstrate the complete lifecycle of the transaction subsystem, a concrete

`CustomerOrder` (ID: `ORD-001`) and a `ReturnOrder` (ID: `RET-001`) are instantiated. The `CustomerOrder` is evaluated against a verified inventory item with an initial stock of 100 units, resulting in a successful deduction of 5 units upon execution. Conversely, the `ReturnOrder` undergoes formal authorization—validated explicitly via a `static_cast` operation to handle specialized order properties—before executing its core processing logic to restore 2 units to the warehouse baseline. This end-to-end integration successfully validates the structural `approve()` to `processOrder()` control flow.

```
testOrders[0] ->processOrder(tmpInv); // CustomerOrder::processOrder()
static_cast<ReturnOrder*>(testOrders[1]) ->a

prove(); // approve first
testOrders[1] ->processOrder(tmpInv); // ReturnOrder::processOrder()
```

8. Finance and Profit Tracking

The `FinanceDept` class tracks revenue and calculates net profit. It is updated automatically every time a customer order is successfully processed.

8.1 Revenue Recording:

Upon the successful execution of `CustomerOrder::processOrder()`, the top-level system automates financial data synchronization by instantly invoking `finance.recordRevenue(order->getRevenue())`. Within this transaction pipeline, the `getRevenue()` member function computes the total line item value based on the underlying algebraic relation of quantity and unit price (`quantity*price_per_unit`). This ensures that realized sales metrics are immediately transferred from the operational inventory state to the financial accounting tracking module without manual intervention.

```
if (order->getStatus() == "Processed") {
    finance.recordRevenue(order->getRevenue());
    procurement.checkReorderLevels(inventory);
}
```

8.2 Profit Calculation:

Net Profit = Total Revenue – Total Employee Salaries – Total Storage Costs

The `showFinanceReport()` method loops over all `Person*` (casting to `Employee*`) for salaries and all `Department*` for storage costs. It then computes and prints the net figure.

Sample Metrics Output Representation:

Total Revenue	\$2500.00
Employee Salaries Overheads	\$453000.00

Storage Slot Operational Costs	\$1000.00
Calculated Net Profit Margin	\$-451500.00

9. Menu Structure and User Interface

9.1 Main Menu

The system features an interactive console-based user interface, divided into a primary operational menu and a departmental sub-menu. The main operations dashboard is configured as follows:

```
=====
FOREIGNS & LOCALS WAREHOUSE MANAGEMENT SYSTEM – MENU
=====
1. Receive Item (Receiving -> Inventory)
2. View Inventory
3. Place Customer Order
4. Place Return Order
5. View All Orders
6. Finance & Profit Report
7. Department Operations
8. Exit
Enter choice:
```

9.2 Department Operations Sub-Menu

Selecting option 7 routes control flow to a robust operational department sub-menu enabling access to individual modules indices metrics:

```
----- DEPARTMENT OPERATIONS -----
1. Receiving Log
2. Quality Control Inspection
3. Storage Location Assignment
4. Order Processing Status
5. Picking & Packing Desk
6. Shipping Dispatch Tracker
7. Procurement Reorder Desk
8. Returns Management
9. HR & Payroll Overview
```

```
10. System Activity Logs (IT)
11. View All Department Summaries
12. Back to Main Menu
Enter department choice:
```

9.3 Menu Implementation Details

The main application interface utilizes a continuous do-while loop paired with a standard switch-case block to handle navigation. To enforce runtime stability, the system incorporates defensive input verification. If an invalid or un-parsable character is entered, the program automatically clears the input stream's error state flags and flushes the buffer, ensuring the application remains responsive without crashing

10. Challenges Encountered and How We Solved Them

Building a multi-layered asset tracking system across seventeen distinct class schemas introduced several technical challenges. Below are the details of the core problems solved during implementation:

Challenge Identified	Solution Applied & Programmatic Logic
Polymorphic Object Slicing and Vectors Allocation	Storing child classes directly inside a standard vector array causes C++ to slice off derived parameters (such as expiryDate), stripping its dynamic binding virtual pointers. We resolved this by changing our collection arrays to hold base pointers on the heap: std::vector<InventoryItem*>. This preserves child components, and manual loops are added to destructors to delete pointers and avoid leaks.
Circular Compile Dependencies	Because Order::processOrder() requires direct parameter references to inventory vectors, while inventory classes look up transaction histories, the preprocessor ran into severe cross-inclusion circular loop errors. We solved this by using Forward Declarations (e.g., class InventoryItem;) in header scopes, moving structural include statements directly into translation files.
Downcasting Constraints	To call child-specific methods like getSalary() on pointers within a generic vector<Person*>, standard calls throw compile errors. We

Challenge Identified	Solution Applied & Programmatic Logic
on Base Pointers Array	used Runtime Type Information (RTTI) via safe dynamic casting: <code>dynamic_cast<Employee*>(p)</code> . This handles type conversions securely, returning <code>nullptr</code> if a customer object is hit, which prevents memory errors.
Console Input Stream Validation Errors	Entering alphabetic characters into option fields triggers stream conversions failbits, causing infinite runtime loop iterations. We wrapped code prompts with automated cleaning guards using <code>cin.clear()</code> and <code>cin.ignore()</code> , flushing invalid inputs and resetting terminal prompts smoothly.
Enforcing Status-Gate Compliance Across Modules	To prevent the processing or shipping of uninspected or failed stock, we added verification steps into <code>CustomerOrder::processOrder()</code> . The system checks if an item's status is marked as <code>Passed</code> , blocking fulfillment and resetting orders to <code>Cancelled</code> if conditions are missed.

11. Collaboration Using Live Share

To streamline development and meet tight project timelines, we set up a collaborative environment using **Visual Studio Code Live Share**. Instead of passing code versions back and forth over email, Moses established an encrypted remote connection relay from his host machine. Michael joined this active session using his verified credentials.

This workflow allows both programmers edit text and use code completion tools simultaneously within the same codebase without overriding each other's changes. Tasks were split by class domains: Moses focused on building structural inventory entities (``Product``, ``InventoryItem``, and derived child classes), while Michael built the person directory trees and transaction logic paths. Sharing an interactive terminal window made testing much more efficient. Michael could enter mock test records directly from his remote console, while Moses watched variable state changes in his local debugger, allowing them to spot and refactor parameter mismatched signatures immediately.

12. Future Improvements

The current platform satisfies all academic assignment requirements, but it runs as a temporary, in-memory simulation. To prepare this architecture for actual corporate use, the team has

outlined three future expansion strategies:

1. **Persistent SQL Database Backends Integration:** Replace volatile in-memory collections with an embedded database connection library like **SQLite** or a server-side **PostgreSQL** cluster. This saves records safely across restarts and protects transaction data from unexpected power loss.
2. **Memory Modernization via Standard Smart Pointers:** Replace traditional raw pointer allocation tracking with managed containers like `std::unique_ptr` and `std::shared_ptr`. This introduces automated object cleanup and removes the need for manual delete calls in destructors, eliminating memory leaks.
3. **Graphical User Interface (GUI) Visual Conversion:** Separate business logic files from the terminal stream and integrate a modern cross-platform user interface framework like **Qt Framework** or **wxWidgets**. This replaces terminal command typing with visual dashboard indicators and forms, making the system much easier to use.

13. Conclusion

The Foreigns & Locals Warehouse Management System demonstrates how object-oriented C++ designs can be used to solve complex, real-world inventory logistics tracking problems. Separating operations into decoupled, interlocking hierarchies ensures high data integrity across all 12 corporate departments.

By leveraging abstraction, encapsulation, inheritance, polymorphism, and reusability, the application remains safe from data corruption errors while staying easy to maintain. This scalable foundation satisfies all assignment criteria, delivering a clean, decoupled class layout that is fully prepared for future enterprise extensions.

C:\Users\ADMIN> OneDrive\Deskop> PROFRAMMING> 2.2> Project 2> programming.cpp WarehouseSystem run()

```
1  /*
2  * =====
3  * FOREIGNS & LOCALS WAREHOUSE MANAGEMENT SYSTEM
4  * JKUAT - EAE2206 Programming for Engineers I
5  * Assignment 3 - Object-Oriented C++ Implementation
6  *
7  * GROUP MEMBERS:
8  *   Moses Chege      (ENM213-0156/2024)
9  *   Michael Muchemi (ENM213-0158/2024)
10 * =====
11 *
12 * DEPARTMENTS (all inherit from Department base class):
13 *   1. Receiving      - accept & log incoming deliveries
14 *   2. Quality Control - inspect items, pass or fail
15 *   3. Inventory Management - track stock levels & locations
16 *   4. Storage/Warehousing - assign & update bin locations
17 *   5. Order Processing - validate & manage customer orders
18 *   6. Picking & Packing - mark orders as packed
19 *   7. Shipping       - dispatch packed orders, generate tracking
20 *   8. Procurement    - flag items below reorder threshold
21 *   9. Returns / Reverse - approve / reject return orders
22 *  10. Administration / HR - staff schedules & payroll
23 *  11. Finance / Accounting - revenue, costs, profit report
24 *  12. IT / Systems    - activity log & system info
25 *
26 * OOP PILARS:
27 *   A) Abstraction    - pure virtual functions (InventoryItem, Person, Order)
28 *   B) Encapsulation  - private/protected data, public interfaces only
29 *   C) Polymorphism   - base-class pointers, overridden virtual methods
30 *   D) Inheritance    - every department inherits from Department;
31 *                       item / person / order hierarchies also apply
32 *   E) Reusability    - shared base classes, vectors of base pointers
33 * =====
34 */
35
36 #include <iostream>
37 #include <string>
38 #include <vector>
39 #include <iomanip>
40 using namespace std;
41
42 // =====
43 // PRODUCT - generates unique ID from item attributes
44 // Format: TYPE-SIZE-COLOUR[-H][-D][-P]
45 // Example: FOOD-M-WHT-P | CHEM-S-CLR-H
46 // =====
47 class Product {
48 private:
49     string name, type, size, colour;
50     bool hazardous, delicate, perishable;
51     string productID;
```

FOREIGN AND LOCALS MANAGEMENT SYSTEM.cpp programming.cpp X Extension: Live Share Mob Timer

C:\Users\ADMIN> OneDrive > Desktop > PROGRAMMING > 2.2 > Project 2 > programming.cpp > WarehouseSystem > run()

```
47 class Product {
48     bool hazardous, delicate, perishable;
49     string productID;
50
51     string generateID() const {
52         string id = type.substr(0,4) + "-" + size.substr(0,1) + "-" + colour.substr(0,3);
53         if (hazardous) id += "-H";
54         if (delicate) id += "-D";
55         if (perishable) id += "-P";
56         return id;
57     }
58
59 public:
60     Product(string name, string type, string size, string colour,
61             bool hazardous, bool delicate, bool perishable)
62         : name(name), type(type), size(size), colour(colour),
63           hazardous(hazardous), delicate(delicate), perishable(perishable)
64     { productID = generateID(); }
65
66     string getProductID() const { return productID; }
67     string getName() const { return name; }
68     bool isHazardous() const { return hazardous; }
69     bool isDelicate() const { return delicate; }
70     bool isPerishable() const { return perishable; }
71 };
72
73 // =====
74 // INVENTORY ITEM - abstract base (Abstraction)
75 // Subclasses: PerishableItem, NonPerishableItem
76 // =====
77 class InventoryItem {
78 protected:
79     Product product;
80     int quantity;
81     string location;
82     double pricePerUnit;
83     string qcStatus; // "Pending" | "Passed" | "Failed"
84     string packStatus; // "Unpacked" | "Packed"
85     string shipStatus; // "In Warehouse" | "Shipped"
86
87 public:
88     InventoryItem(string name, string type, string size, string colour,
89                 bool hazardous, bool delicate, bool perishable,
90                 int qty, string loc, double price)
91         : product(name, type, size, colour, hazardous, delicate, perishable),
92           quantity(qty), location(loc), pricePerUnit(price),
93           qcStatus("Pending"), packStatus("Unpacked"), shipStatus("In Warehouse") {}
94
95     virtual ~InventoryItem() {}
96
97     virtual string getItemType() const = 0; // pure virtual - Abstraction
```

C:\Users\ADMIN > OneDrive > Desktop > PROGRAMMING > 2.2 > Project 2 > programming.cpp WarehouseSystem > run()

```
79 class InventoryItem {
98
99     virtual string getItemType() const = 0;    // pure virtual - Abstraction
100
101     virtual void displayItem() const {
102         cout << "----Inventory Status----" << endl;
103         cout << "Type:      " << getItemType() << endl;
104         cout << "Name:      " << product.getName() << endl;
105         cout << "ID:        " << product.getProductID() << endl;
106         cout << "Stock:     " << quantity << endl;
107         cout << "Location:  " << location << endl;
108         cout << "Price/unit: $" << fixed << setprecision(2) << pricePerUnit << endl;
109         cout << "Hazardous:  " << (product.isHazardous() ? "Yes" : "No") << endl;
110         cout << "Perishable: " << (product.isPerishable() ? "Yes" : "No") << endl;
111         cout << "Fragile:    " << (product.isDelicate() ? "Yes" : "No") << endl;
112         cout << "QC Status:  " << qcStatus << endl;
113         cout << "Ship Status:" << shipStatus << endl;
114         cout << "-----" << endl;
115     }
116
117     // Getters / Setters (Encapsulation)
118     string getProductID() const { return product.getProductID(); }
119     string getName() const { return product.getName(); }
120     int getQuantity() const { return quantity; }
121     double getPricePerUnit() const { return pricePerUnit; }
122     string getQCStatus() const { return qcStatus; }
123     string getLocation() const { return location; }
124
125     void setQuantity(int q) { quantity = q; }
126     void setLocation(string loc) { location = loc; }
127     void setQCStatus(string s) { qcStatus = s; }
128     void setPackStatus(string s) { packStatus = s; }
129     void setShipStatus(string s) { shipStatus = s; }
130 };
131
132 // ---- Perishable Item (Inheritance from InventoryItem) ----
133 class PerishableItem : public InventoryItem {
134 private:
135     string expiryDate;
136 public:
137     PerishableItem(string name, string type, string size, string colour,
138         bool hazardous, bool delicate, int qty, string loc,
139         double price, string expiry)
140         : InventoryItem(name, type, size, colour, hazardous, delicate,
141             true, qty, loc, price),
142         expiryDate(expiry) {}
143
144     string getItemType() const override { return "Perishable"; }
145
146     void displayItem() const override {
147         InventoryItem::displayItem();    // pure base (PerishableItem)
```

C:\Users\ADMIN> OneDrive\ Desktop > PROFAMMING > 2.2 > Project 2 > G:\ programming.cpp > WarehouseSystem > run()

```
133 class PerishableItem : public InventoryItem {
134
135     void displayItem() const override {
136         InventoryItem::displayItem();           // reuse base (Reusability)
137         cout << "Expiry Date: " << expiryDate << endl;
138         cout << "-----" << endl;
139     }
140 };
141
142 // ---- Non-Perishable Item (Inheritance from InventoryItem) ----
143 class NonPerishableItem : public InventoryItem {
144 public:
145     NonPerishableItem(string name, string type, string size, string colour,
146                        bool hazardous, bool delicate, int qty, string loc, double price)
147         : InventoryItem(name, type, size, colour, hazardous, delicate,
148                        false, qty, loc, price) {}
149
150     string getItemType() const override { return "Non-Perishable"; }
151 };
152
153 // =====
154 // PERSON - abstract base (Abstraction)
155 // Subclasses: Employee, Customer
156 // =====
157 class Person {
158 protected:
159     int id;
160     string name, contact;
161 public:
162     Person(int id, string name, string contact)
163         : id(id), name(name), contact(contact) {}
164
165     virtual ~Person() {}
166     virtual string getRole() const = 0;
167     virtual void display() const = 0;
168
169     string getName() const { return name; }
170     int getID() const { return id; }
171 };
172
173 // ---- Employee (Inheritance from Person) ----
174 class Employee : public Person {
175 private:
176     string department, role;
177     double salary;
178 public:
179     Employee(int id, string name, string contact,
180             string department, double salary, string role)
181         : Person(id, name, contact),
182         department(department), role(role), salary(salary) {}
183
184 }
```

C:\Users\ADMIN > OneDrive > Desktop > PROGRAMMING > 2.2 > Project 2 > C:\programming.cpp > Customer > Customer(int, string, string, string, double)

```
185 class Employee : public Person {
194
195     string getRole() const override { return role; }
196     string getDepartment() const { return department; }
197     double getSalary() const { return salary; }
198
199     void display() const override {
200         cout << "ID: " << id
201             << " | " << name
202             << " | Role: " << role
203             << " | Dept: " << department
204             << " | Salary: $" << fixed << setprecision(2) << salary << endl;
205     }
206 };
207
208 // ---- Customer (Inheritance from Person) ----
209 class Customer : public Person {
210 private:
211     string address;
212     double creditLimit;
213 public:
214     Customer(int id, string name, string contact, string address, double creditLimit)
215         : Person(id, name, contact), address(address), creditLimit(creditLimit) {}
216
217     string getRole() const override { return "Customer"; }
218
219     void display() const override {
220         cout << "Customer: " << name
221             << " | Contact: " << contact
222             << " | Address: " << address
223             << " | Credit: $" << fixed << setprecision(2) << creditLimit << endl;
224     }
225 };
226
227 // =====
228 // ORDER - abstract base (Abstraction)
229 // Subclasses: CustomerOrder, ReturnOrder
230 // =====
231 class Order {
232 protected:
233     string orderID, productID, date, status;
234     int quantity;
235 public:
236     Order(string orderID, string productID, string date, int quantity)
237         : orderID(orderID), productID(productID), date(date),
238           status("Pending"), quantity(quantity) {}
239
240     virtual ~Order() {}
241     virtual void processOrder(vector<InventoryItem*>& inv) = 0;
242     virtual void displayOrder() const = 0;
243 }
```

C:\Users\ADMIN> OneDrive > Desktop > PROGRAMMING > 2.2 > Project 2 > programming.cpp > CustomerOrder > CustomerOrder(string, string, string, int, string, string, double)

```
231 class Order {
232     // void displayOrder() const { }
233 };
234
235 string getOrderID() const { return orderID; }
236 string getProductID() const { return productID; }
237 string getStatus() const { return status; }
238 int getQuantity() const { return quantity; }
239 virtual double getRevenue() const { return 0; }
240 };
241
242 // ---- Customer Order (Inheritance from Order) ----
243 class CustomerOrder : public Order {
244 private:
245     string customerName, deliveryAddress;
246     double totalCost;
247 public:
248     CustomerOrder(string oID, string pID, string date, int qty,
249                 string cust, string addr, double price)
250         : Order(oID, pID, date, qty),
251           customerName(cust), deliveryAddress(addr),
252           totalCost(qty * price) {}
253
254     void processOrder(vector<InventoryItem*>& inv) override {
255         for (auto item : inv) {
256             if (item->getProductID() == productID) {
257                 if (item->getQCStatus() != "Passed") {
258                     cout << "[Order Processing] Item has not passed QC yet!" << endl;
259                     status = "Cancelled"; return;
260                 }
261                 if (item->getQuantity() >= quantity) {
262                     item->setQuantity(item->getQuantity() - quantity);
263                     status = "Processed";
264                     cout << "[Order Processing] SUCCESS | Dispatching " << quantity
265                         << " unit(s) of " << item->getName()
266                         << " to " << deliveryAddress
267                         << ". Remaining stock: " << item->getQuantity() << endl;
268                 } else {
269                     status = "Cancelled";
270                     cout << "[Order Processing] FAILED | Insufficient stock for: "
271                         << productID << endl;
272                 }
273             }
274             return;
275         }
276     }
277
278     cout << "[Order Processing] ERROR | Product ID not found: " << productID << endl;
279 }
280
281 void displayOrder() const override {
282     cout << "-----" << endl;
283     cout << "[Customer Order]" << endl;
284     cout << "Order ID: " << orderID << endl;
285 }
```

C:\Users\ADMIN\OneDrive\Desktop\PROFRAMMING\2.2\Project 2\programming.cpp ReturnOrder

```
252 class CustomerOrder : public Order {
288 void displayOrder() const override {
291     cout << "Order ID: " << orderID << endl;
292     cout << "Date: " << date << endl;
293     cout << "Product ID: " << productID << endl;
294     cout << "Quantity: " << quantity << endl;
295     cout << "Customer: " << customerName << endl;
296     cout << "Deliver to: " << deliveryAddress << endl;
297     cout << "Total Cost: $" << fixed << setprecision(2) << totalCost << endl;
298     cout << "Status: " << status << endl;
299 }
300
301 double getRevenue() const override { return totalCost; }
302 string getAddress() const { return deliveryAddress; }
303 };
304
305 // ---- Return Order (Inheritance from Order) ----
306 class ReturnOrder : public Order {
307 private:
308     string returnReason, approvalStatus;
309 public:
310     ReturnOrder(string oID, string pID, string date, int qty, string reason)
311         : Order(oID, pID, date, qty),
312           returnReason(reason), approvalStatus("Pending") {}
313
314     void approve() {
315         approvalStatus = "Approved";
316         cout << "[Returns] Order " << orderID << " APPROVED." << endl;
317     }
318     void reject() {
319         approvalStatus = "Rejected";
320         status = "Cancelled";
321         cout << "[Returns] Order " << orderID << " REJECTED." << endl;
322     }
323
324     void processOrder(vector<InventoryItem*>& inv) override {
325         if (approvalStatus != "Approved") {
326             cout << "[Returns] Cannot process - approval status: "
327                 << approvalStatus << endl;
328             return;
329         }
330         for (auto item : inv) {
331             if (item->getProductID() == productID) {
332                 item->setQuantity(item->getQuantity() + quantity);
333                 status = "Processed";
334                 cout << "[Returns] Stock restored | " << quantity
335                     << " units returned. New stock: "
336                     << item->getQuantity() << endl;
337                 return;
338             }
339         }
340     }
341 }
```

C: > Users > ADMIN > OneDrive > Desktop > PROFRAMMING > 2.2 > Project 2 > programming.cpp > ReturnOrder

```
306 class ReturnOrder : public Order {
324     void processOrder(vector<InventoryItem*>& inv) override {
330         for (auto item : inv) {
331             if (item->getProductID() == productID) {
338             }
339         }
340         cout << "[Returns] ERROR | Product ID not found: " << productID << endl;
341     }
342
343     void displayOrder() const override {
344         cout << "-----" << endl;
345         cout << "[Return Order]" << endl;
346         cout << "Order ID: " << orderID << endl;
347         cout << "Date: " << date << endl;
348         cout << "Product ID: " << productID << endl;
349         cout << "Quantity: " << quantity << endl;
350         cout << "Return Reason: " << returnReason << endl;
351         cout << "Approval Status: " << approvalStatus << endl;
352         cout << "Order Status: " << status << endl;
353     }
354
355     string getApprovalStatus() const { return approvalStatus; }
356 };
357
358 // =====
359 // DEPARTMENT - base class (Inheritance root for all 12 depts)
360 // =====
361 class Department {
362 protected:
363     string name;
364     vector<Employee*> staff;
365     double storageCost;
366 public:
367     Department(string name, double storageCost = 0.0)
368         : name(name), storageCost(storageCost) {}
369
370     virtual ~Department() {}
371
372     void addEmployee(Employee* e) { staff.push_back(e); }
373
374     double getTotalSalaries() const {
375         double t = 0;
376         for (auto e : staff) t += e->getSalary();
377         return t;
378     }
379     double getStorageCost() const { return storageCost; }
380     string getName() const { return name; }
381
382     virtual void displaySummary() const {
383         cout << "Dept: " << name
```

C:\Users\ADMIN> OneDrive > Desktop > PROFRAMMING > 2.2 > Project 2 > programming.cpp > ReturnOrder

```

361 class Department {
362     virtual void displaySummary() const {
363         cout << "Dept: " << name
364             << " | Staff: " << staff.size()
365             << " | Salaries: $" << fixed << setprecision(2) << getTotalSalaries()
366             << " | Storage: $" << storageCost << endl;
367     }
368 };
369
370 // =====
371 // DEPARTMENT SUBCLASSES (Inheritance from Department)
372 // Each adds only what is specific to that department.
373 // =====
374
375 // 1. RECEIVING - logs every delivery that comes in
376 class ReceivingDept : public Department {
377 private:
378     vector<string> deliveryLog;
379 public:
380     ReceivingDept() : Department("Receiving") {}
381
382     // Called when a new item enters the warehouse
383     void logDelivery(const string& itemName, const string& itemID, int qty) {
384         string entry = "Item: " + itemName + " (ID: " + itemID + ") x" + to_string(qty);
385         deliveryLog.push_back(entry);
386         cout << "[Receiving] Delivery received & logged. Sending to Quality Control." << endl;
387     }
388
389     void showDeliveryLog() const {
390         cout << "\n[Receiving] Delivery Log (" << deliveryLog.size() << " entries):" << endl;
391         if (deliveryLog.empty()) { cout << " No deliveries yet." << endl; return; }
392         for (size_t i = 0; i < deliveryLog.size(); i++)
393             cout << " " << (i+1) << ". " << deliveryLog[i] << endl;
394     }
395
396     void displaySummary() const override {
397         cout << "Dept: Receiving | Deliveries logged: " << deliveryLog.size() << endl;
398     }
399 };
400
401 // 2. QUALITY CONTROL - inspect items, mark pass or fail
402 class QCDept : public Department {
403 private:
404     int passed, failed;
405 public:
406     QCDept() : Department("Quality Control"), passed(0), failed(0) {}
407
408     // Inspect a single item from the inventory by ID
409     void inspectItem(vector<InventoryItem*>& inventory) {
410         if (inventory.empty()) { cout << "[QC] No items in inventory." << endl; return; }
411         cout << "\n[QC] Items pending inspection:" << endl;

```

C: > Users > ADMIN > OneDrive > Desktop > PROFRRAMMING > 2.2 > Project 2 > G: programming.cpp > ReturnOrder

```
422 class QCDept : public Department {
429     void inspectItem(vector<InventoryItem*>& inventory) {
432         bool any = false;
433         for (auto item : inventory) {
434             if (item->getQCStatus() == "Pending") {
435                 cout << " ID: " << item->getProductID()
436                     << " | Name: " << item->getName() << endl;
437                 any = true;
438             }
439         }
440         if (!any) { cout << " All items already inspected." << endl; return; }
441
442         string id;
443         cout << "Enter Product ID to inspect: "; cin >> id;
444         int result;
445         cout << "Inspection result - 1=Pass, 0=Fail: "; cin >> result;
446
447         for (auto item : inventory) {
448             if (item->getProductID() == id) {
449                 if (result == 1) {
450                     item->setQCStatus("Passed");
451                     passed++;
452                     cout << "[QC] " << item->getName() << " - PASSED. Cleared for storage." << endl;
453                 } else {
454                     item->setQCStatus("Failed");
455                     failed++;
456                     cout << "[QC] " << item->getName() << " - FAILED. Item quarantined." << endl;
457                 }
458                 return;
459             }
460         }
461         cout << "[QC] Product ID not found." << endl;
462     }
463
464     void displaySummary() const override {
465         cout << "Dept: Quality Control | Passed: " << passed << " | Failed: " << failed << endl;
466     }
467 };
468
469 // 3. STORAGE / WAREHOUSING - assign or update bin/aisle locations
470 class StorageDept : public Department {
471 private:
472     double storageCostPerSlot;
473 public:
474     StorageDept(double costPerSlot = 500.0)
475         : Department("Storage/Warehousing", 0.0),
476           storageCostPerSlot(costPerSlot) {}
477
478     void assignLocation(vector<InventoryItem*>& inventory) {
479         if (inventory.empty()) { cout << "[Storage] Inventory is empty." << endl; return; }
480         cout << "\n[Storage] Current locations:" << endl;
```

C:\> Users > ADMIN > OneDrive > Desktop > PROFRAMMING > 2.2 > Project 2 > programming.cpp > OrderProcessingDept

```
470 class StorageDept : public Department {
471     void assignLocation(vector<InventoryItem*>& inventory) {
472         cout << "\n[Storage] Current locations:" << endl;
473         for (auto item : inventory)
474             cout << " ID: " << item->getProductID()
475                 << " | Name: " << item->getName()
476                 << " | Location: " << item->getLocation() << endl;
477
478         string id, newLoc;
479         cout << "Enter Product ID to update location: "; cin >> id;
480         cout << "New location (e.g. Aisle-3, Fridge-B2): "; cin >> newLoc;
481
482         for (auto item : inventory) {
483             if (item->getProductID() == id) {
484                 item->setLocation(newLoc);
485                 storageCost += storageCostPerSlot;
486                 cout << "[Storage] Location updated to " << newLoc << endl;
487                 return;
488             }
489         }
490         cout << "[Storage] Product ID not found." << endl;
491     }
492
493     void displaySummary() const override {
494         cout << "Dept: Storage/Warehousing | Storage Cost: $"
495             << fixed << setprecision(2) << storageCost << endl;
496     }
497 };
498
499 // 4. ORDER PROCESSING - validates orders before they are fulfilled
500 // (actual order creation is in CustomerOrder; this dept checks/reports)
501 class OrderProcessingDept : public Department {
502 public:
503     OrderProcessingDept() : Department("Order Processing") {}
504
505     void showOrderStatus(const vector<Order*>& orders) const {
506         cout << "\n[Order Processing] Order Status Report:" << endl;
507         if (orders.empty()) { cout << " No orders placed yet." << endl; return; }
508         for (auto o : orders) {
509             cout << " Order: " << o->getOrderID()
510                 << " | Status: " << o->getStatus() << endl;
511         }
512     }
513
514     void displaySummary() const override {
515         cout << "Dept: Order Processing | Validates customer orders before fulfilment." << endl;
516     }
517 };
518
519 // 5. PICKING & PACKING - marks ordered items as packed for shipment
520 class PickingPackingDept : public Department {
```

C:\Users\ADMIN> OneDrive > Desktop > PROGRAMMING > 2.2 > Project 2 > G:\programming.cpp > OrderProcessingDept

```
528 class PickingPackingDept : public Department {
529 private:
530     int packedCount;
531 public:
532     PickingPackingDept() : Department("Picking & Packing"), packedCount(0) {}
533
534     void packOrder(vector<Order*>& orders, vector<InventoryItem*>& inventory) {
535         if (orders.empty()) { cout << "[Picking & Packing] No orders to pack." << endl; return; }
536         cout << "\n[Picking & Packing] Processed orders available to pack:" << endl;
537         bool any = false;
538         for (auto o : orders) {
539             if (o->getStatus() == "Processed") {
540                 cout << " " << o->getOrderID() << " | Product: " << o->getProductID() << endl;
541                 any = true;
542             }
543         }
544         if (!any) { cout << " No processed orders found." << endl; return; }
545
546         string oID;
547         cout << "Enter Order ID to pack: "; cin >> oID;
548         for (auto o : orders) {
549             if (o->getOrderID() == oID && o->getStatus() == "Processed") {
550                 // Mark the inventory item as packed
551                 for (auto item : inventory)
552                     if (item->getProductID() == o->getProductID())
553                         item->setPackStatus("Packed");
554                 packedCount++;
555                 cout << "[Picking & Packing] Order " << oID
556                     << " packed and ready for shipping." << endl;
557                 return;
558             }
559         }
560         cout << "[Picking & Packing] Order not found or not yet processed." << endl;
561     }
562
563     void displaySummary() const override {
564         cout << "Dept: Picking & Packing | Orders packed: " << packedCount << endl;
565     }
566 };
567
568 // 6. SHIPPING - dispatches packed orders; generates tracking numbers
569 class ShippingDept : public Department {
570 private:
571     int shippedCount;
572     vector<string> shippingLog;
573 public:
574     ShippingDept() : Department("Shipping"), shippedCount(0) {}
575
576     void dispatchOrder(vector<Order*>& orders, vector<InventoryItem*>& inventory) {
577         if (orders.empty()) { cout << "[Shipping] No orders available." << endl; return; }
578         cout << "\n[Shipping] Orders ready to dispatch:" << endl;
```

C:\Users\ADMIN> OneDrive > Desktop > PROGRAMMING > 2.2 > Project 2 > programming.cpp > OrderProcessingDept

```
569 class ShippingDept : public Department {
576 void dispatchOrder(vector<Order*>& orders, vector<InventoryItem*>& inventory) {
578     cout << "\n[Shipping] Orders ready to dispatch:" << endl;
579     bool any = false;
580     for (auto o : orders) {
581         if (o->getStatus() == "Processed") {
582             cout << " " << o->getOrderID() << " | Product: " << o->getProductID() << endl;
583             any = true;
584         }
585     }
586     if (!any) { cout << " No orders ready for shipping." << endl; return; }
587
588     string oID;
589     cout << "Enter Order ID to dispatch: "; cin >> oID;
590     for (auto o : orders) {
591         if (o->getOrderID() == oID && o->getStatus() == "Processed") {
592             // Mark item as shipped
593             for (auto item : inventory)
594                 if (item->getProductID() == o->getProductID())
595                     item->setShipStatus("Shipped");
596
597             string tracking = "TRK-" + to_string(100 + shippedCount);
598             shippedCount++;
599             shippingLog.push_back("Order: " + oID + " | Tracking: " + tracking);
600             cout << "[Shipping] Order " << oID
601                 << " dispatched. Tracking No: " << tracking << endl;
602             return;
603         }
604     }
605     cout << "[Shipping] Order not found or not processed." << endl;
606 }
607
608 void showShippingLog() const {
609     cout << "\n[Shipping Log]: " << endl;
610     if (shippingLog.empty()) { cout << " Nothing shipped yet." << endl; return; }
611     for (auto& s : shippingLog) cout << " " << s << endl;
612 }
613
614 void displaySummary() const override {
615     cout << "Dept: Shipping | Orders dispatched: " << shippedCount << endl;
616 }
617 };
618
619 // 7. PROCUREMENT - flags items below a reorder threshold; raises purchase requests
620 class ProcurementDept : public Department {
621 private:
622     int reorderThreshold;
623     vector<string> purchaseRequests;
624 public:
625     ProcurementDept(int threshold = 10)
626         : Department("Procurement"), reorderThreshold(threshold) {}
```

```

FOREIGN AND LOCALS MANAGEMENT SYSYTEM.cpp  programming.cpp X  Extension: Live Share Mob Timer
C:\> Users > ADMIN > OneDrive > Desktop > PROFGRAMMING > 2.2 > Project 2 > programming.cpp > ProcurementDept > showPurchaseRequests() const
620 class ProcurementDept : public Department {
621     ProcurementDept(int threshold = 10)
622     : Department("Procurement"), reorderThreshold(threshold) {}
623
624     void checkReorderLevels(const vector<InventoryItem*> & inventory) {
625         cout << "\n[Procurement] Checking reorder levels (threshold = "
626             << reorderThreshold << " units):" << endl;
627         bool any = false;
628         for (auto item : inventory) {
629             if (item->getQuantity() <= reorderThreshold) {
630                 string req = "REORDER: " + item->getName()
631                     + " (ID: " + item->getProductID()
632                     + ") - only " + to_string(item->getQuantity()) + " left";
633                 purchaseRequests.push_back(req);
634                 cout << " [!] " << req << endl;
635                 any = true;
636             }
637         }
638         if (!any) cout << " All stock levels are adequate." << endl;
639     }
640
641     void showPurchaseRequests() const {
642         cout << "\n[Procurement] Purchase Requests:" << endl;
643         if (purchaseRequests.empty()) { cout << " No requests raised." << endl; return; }
644         for (size_t i = 0; i < purchaseRequests.size(); i++)
645             cout << " " << (i+1) << ". " << purchaseRequests[i] << endl;
646     }
647
648     void displaySummary() const override {
649         cout << "Dept: Procurement | Purchase requests raised: "
650             << purchaseRequests.size() << endl;
651     }
652 };
653
654 // 8. RETURNS / REVERSE LOGISTICS - approve or reject return orders
655 class ReturnsDept : public Department {
656 private:
657     int approved, rejected;
658 public:
659     ReturnsDept() : Department("Returns/Reverse Logistics"), approved(0), rejected(0) {}
660
661     void processReturn(vector<Order*> & orders, vector<InventoryItem*> & inventory) {
662         cout << "\n[Returns] Pending return orders:" << endl;
663         bool any = false;
664         for (auto o : orders) {
665             auto ret = dynamic_cast<ReturnOrder*>(o);
666             if (ret && ret->getApprovalStatus() == "Pending") {
667                 cout << " " << ret->getOrderID()
668                     << " | Product: " << ret->getProductID() << endl;
669                 any = true;
670             }
671         }
672     }
673

```

```

FOREIGN AND LOCALS MANAGEMENT SYSYTEM.cpp  programming.cpp X  Extension: Live Share Mob Timer
C:\Users\ADMIN> OneDrive > Desktop > PROFRAMMING > 2.2 > Project 2 > programming.cpp > ProcurementDept > showPurchaseRequests() const
659 class ReturnsDept : public Department {
665     void processReturn(vector<Order*>& orders, vector<InventoryItem*>& inventory) {
668         for (auto o : orders) {
670             if (ret && ret->getApprovalStatus() == "Pending") {
674             }
675         }
676         if (!any) { cout << " No pending returns." << endl; return; }
677
678         string oID;
679         cout << "Enter Return Order ID: "; cin >> oID;
680         int decision;
681         cout << "Decision - 1=Approve, 0=Reject: "; cin >> decision;
682
683         for (auto o : orders) {
684             auto ret = dynamic_cast<ReturnOrder*>(o);
685             if (ret && ret->getOrderID() == oID) {
686                 if (decision == 1) {
687                     ret->approve();
688                     ret->processOrder(inventory);
689                     approved++;
690                 } else {
691                     ret->reject();
692                     rejected++;
693                 }
694             }
695         }
696     }
697     cout << "[Returns] Order ID not found." << endl;
698 }
699
700 void displaySummary() const override {
701     cout << "Dept: Returns | Approved: " << approved
702     << " | Rejected: " << rejected << endl;
703 }
704 };
705
706 // 9. ADMINISTRATION / HR - manages staff records and payroll summary
707 class HRDept : public Department {
708 public:
709     HRDept() : Department("Administration/HR") {}
710
711     void showAllStaff(const vector<Person*>& people) const {
712         cout << "\n[HR] Staff & Customer Directory:" << endl;
713         if (people.empty()) { cout << " No records." << endl; return; }
714         for (auto p : people) p->display(); // Polymorphism
715     }
716
717     void showPayroll(const vector<Person*>& people) const {
718         cout << "\n[HR] Payroll Summary:" << endl;
719         double total = 0;
720         for (auto p : people) {

```

C:\Users\ADMIN> OneDrive\Deskto > PROFRAMMING > 2.2 > Project 2 > programming.cpp > ProcurementDept > showPurchaseRequests() const

```
707 class HRDept : public Department {
717     void showPayroll(const vector<Person*>& people) const {
720         for (auto p : people) {
721             auto e = dynamic_cast<Employee*>(p);
722             if (e) {
723                 cout << " " << e->getName()
724                     << " | " << e->getRole()
725                     << " | $" << fixed << setprecision(2) << e->getSalary() << endl;
726                 total += e->getSalary();
727             }
728         }
729         cout << " TOTAL PAYROLL: $" << fixed << setprecision(2) << total << endl;
730     }
731
732     void displaySummary() const override {
733         cout << "Dept: Administration/HR | Manages staff schedules & payroll." << endl;
734     }
735 };
736
737 // 10. FINANCE / ACCOUNTING - revenue, costs, profit (Inheritance + extra logic)
738 class FinanceDept : public Department {
739 private:
740     double revenue;
741 public:
742     FinanceDept() : Department("Finance/Accounting"), revenue(0.0) {}
743
744     void recordRevenue(double amount) { revenue += amount; }
745     double getRevenue() const { return revenue; }
746
747     double getProfit(double salaries, double storageCosts) const {
748         return revenue - salaries - storageCosts;
749     }
750
751     void showFinanceReport(const vector<Person*>& people,
752                           const vector<Department*>& depts) const {
753         double salaries = 0;
754         for (auto p : people)
755             if (auto e = dynamic_cast<Employee*>(p)) salaries += e->getSalary();
756
757         double storage = 0;
758         for (auto d : depts) storage += d->getStorageCost();
759
760         double profit = getProfit(salaries, storage);
761
762         cout << "\n===== Finance & Profit Report =====" << endl;
763         cout << "Total Revenue:   $" << fixed << setprecision(2) << revenue << endl;
764         cout << "Employee Salaries: $" << salaries << endl;
765         cout << "Storage Costs:     $" << storage << endl;
766         cout << "-----" << endl;
767         cout << "Net Profit:        $" << profit << endl;
768         cout << "===== " << endl;
```

C: > Users > ADMIN > OneDrive > Desktop > PROFRAMMING > 2.2 > Project 2 > programming.cpp > ProcurementDept > showPurchaseRequests() const

```
738 class FinanceDept : public Department {
752     const vector<Department*>& depts) const {
768         cout << "===== " << endl;
769     }
770
771     void displaySummary() const override {
772         cout << "Dept: Finance | Revenue: $" << fixed << setprecision(2) << revenue << endl;
773     }
774 };
775
776 // 11. IT / SYSTEMS - activity log, system information
777 class ITDept : public Department {
778 private:
779     vector<string> activityLog;
780 public:
781     ITDept() : Department("IT/Systems") {}
782
783     void log(const string& entry) {
784         activityLog.push_back(entry);
785     }
786
787     void showLog() const {
788         cout << "\n[IT/Systems] Activity Log (" << activityLog.size() << " entries):" << endl;
789         if (activityLog.empty()) { cout << " No activity recorded." << endl; return; }
790         for (size_t i = 0; i < activityLog.size(); i++)
791             cout << " " << (i+1) << ". " << activityLog[i] << endl;
792     }
793
794     void showSystemInfo() const {
795         cout << "\n[IT/Systems] System Info:" << endl;
796         cout << " System: Foreigns & Locals WMS v2.0" << endl;
797         cout << " Lang: C++17" << endl;
798         cout << " Status: Online" << endl;
799         cout << " Logs: " << activityLog.size() << " entries" << endl;
800     }
801
802     void displaySummary() const override {
803         cout << "Dept: IT/Systems | Log entries: " << activityLog.size() << endl;
804     }
805 };
806
807 // =====
808 // WAREHOUSE SYSTEM - ties all departments and entities together
809 // =====
810 class WarehouseSystem {
811 private:
812     // ---- data stores ----
813     vector<InventoryItem*> inventory;
814     vector<Person*> people;
815     vector<Order*> orders;
816
```

FOREIGN AND LOCALS MANAGEMENT SYSYTEM.cpp programming.cpp Extension: Live Share Mob Timer

C: > Users > ADMIN > OneDrive > Desktop > PROGRAMMING > 2.2 > Project 2 > programming.cpp > ProcurementDept > showPurchaseRequests() const

```
810 class WarehouseSystem {
816
817     // ---- all 12 departments ----
818     ReceivingDept    receiving;
819     QCDept           qc;
820     StorageDept      storage;
821     OrderProcessingDept orderProcessing;
822     PickingPackingDept pickingPacking;
823     ShippingDept     shipping;
824     ProcurementDept  procurement;
825     ReturnsDept      returnsDept;
826     HRDept           hr;
827     FinanceDept      finance;
828     ITDept           it;
829     // (Department base used for the vector below)
830     vector<Department*> allDepts; // non-owning pointers for summary
831
832     int orderCounter = 1;
833     int returnCounter = 1;
834
835     // ----- OOP Checkpoint demos -----
836     void runCheckpoints() {
837         cout << "\n===== " << endl;
838         cout << " CHECKPOINT A: InventoryItem Hierarchy " << endl;
839         cout << " Polymorphism via InventoryItem* ptr " << endl;
840         cout << "===== " << endl;
841         vector<InventoryItem*> demo;
842         demo.push_back(new PerishableItem("Maize Flour", "FOOD", "M", "WHT",
843                                           false, false, 100, "Fridge-A1", 45.0, "2026-12-01"));
844         demo.push_back(new NonPerishableItem("Battery Acid", "CHEM", "S", "CLR",
845                                              true, false, 50, "Aisle-B3", 120.0));
846         for (auto i : demo) { i->setQCStatus("Passed"); i->displayItem(); }
847         for (auto i : demo) delete i;
848
849         cout << "\n===== " << endl;
850         cout << " CHECKPOINT B: Person Hierarchy " << endl;
851         cout << " Polymorphism via Person* ptr " << endl;
852         cout << "===== " << endl;
853         vector<Person*> persons;
854         persons.push_back(new Customer(1, "Wanjiku Njoroge", "0712345678", "Githurai 45", 10000));
855         persons.push_back(new Employee(2, "Otieno Mwangi", "0723456789", "Management", 80000, "Manager"));
856         persons.push_back(new Employee(3, "James Kamau", "0734567890", "Quality Control", 60000, "QC Officer"));
857         for (auto p : persons) p->display();
858         for (auto p : persons) delete p;
859
860         cout << "\n===== " << endl;
861         cout << " CHECKPOINT C: Order Hierarchy " << endl;
862         cout << " Polymorphism via Order* ptr " << endl;
863         cout << "===== " << endl;
864         vector<InventoryItem*> tmpInv;
```

C: > Users > ADMIN > OneDrive > Desktop > PROFRAMMING > 2.2 > Project 2 > programming.cpp > ProcurementDept > showPurchaseRequests() const

```
810 class WarehouseSystem {
836     void runCheckpoints() {
865         tmpInv.push_back(new NonPerishableItem("Demo Item", "FOOD", "M", "WHT",
866                                               false, false, 100, "Aisle-1", 50.0));
867         tmpInv[0]->setQCStatus("Passed");
868
869         vector<Order*> testOrders;
870         testOrders.push_back(new CustomerOrder("ORD-001", tmpInv[0]->getProductID(),
871                                               "2026-05-01", 5,
872                                               "Wanjiku Njoroge", "Githurai 45", 50.0));
873         testOrders.push_back(new ReturnOrder("RET-001", tmpInv[0]->getProductID(),
874                                              "2026-05-02", 2, "Damaged on arrival"));
875         testOrders[0]->processOrder(tmpInv);
876         static_cast<ReturnOrder*>(testOrders[1])->approve();
877         testOrders[1]->processOrder(tmpInv);
878         for (auto o : testOrders) { o->displayOrder(); delete o; }
879         for (auto i : tmpInv) delete i;
880     }
881
882     // ----- Load dummy staff -----
883     void loadDummyData() {
884         auto mgr = new Employee(101, "Alice Kamau", "0700111001", "Admin", 90000, "Manager");
885         auto qcOff = new Employee(102, "James Kamau", "0700111002", "QC", 60000, "QC Officer");
886         auto store = new Employee(103, "Bob Ochieng", "0700111003", "Inventory", 50000, "Storekeeper");
887         auto picker = new Employee(104, "Aisha Omondi", "0700111004", "Picking", 45000, "Picker");
888         auto driver = new Employee(105, "Brian Ouma", "0700111005", "Shipping", 48000, "Driver");
889         auto buyer = new Employee(106, "Grace Wangari", "0700111006", "Procurement", 55000, "Buyer");
890         auto fin = new Employee(107, "Carol Njeri", "0700111007", "Finance", 65000, "Accountant");
891
892         auto cust1 = new Customer(201, "Wanjiku Njoroge", "0712345678", "Githurai 45", 10000);
893         auto cust2 = new Customer(202, "Otieno Mwangi", "0723456789", "Industrial Area", 25000);
894
895         people.push_back(mgr); people.push_back(qcOff); people.push_back(store);
896         people.push_back(picker); people.push_back(driver); people.push_back(buyer);
897         people.push_back(fin); people.push_back(cust1); people.push_back(cust2);
898
899         // Assign staff to departments
900         hr.addEmployee(mgr);
901         qc.addEmployee(qcOff);
902         storage.addEmployee(store);
903         pickingPacking.addEmployee(picker);
904         shipping.addEmployee(driver);
905         procurement.addEmployee(buyer);
906         finance.addEmployee(fin);
907
908         it.log("System started.");
909         it.log("Dummy staff and customers loaded.");
910     }
911
912     // -----
913     // NEW ACTIONS
```

```
810 class WarehouseSystem {
912 // -----
913 // MENU ACTIONS
914 // -----
915
916 // 1. Receive & Add Item (Receiving → QC pending → Inventory)
917 void receiveItem() {
918     string name, type, size, colour, location, expiry;
919     int qty, tmp;
920     bool hazardous, delicate, perishable;
921     double price;
922
923     cout << "\n--[Receiving Dept] Enter Delivery Details--" << endl;
924     cout << "Name: "; cin >> ws; getline(cin, name);
925     cout << "Type (FOOD/ELEC/CHEM): "; cin >> type;
926     cout << "Size (S/M/L): "; cin >> size;
927     cout << "Colour: "; cin >> colour;
928     cout << "Hazardous? (1=Yes,0=No): ";
929     while (!(cin >> tmp) || (tmp != 0 && tmp != 1)) {
930         cout << "Invalid. Enter 1 or 0: "; cin.clear(); cin.ignore(1000, '\n');
931     }
932     hazardous = tmp;
933     cout << "Perishable? (1=Yes,0=No): "; cin >> tmp; perishable = tmp;
934     cout << "Fragile? (1=Yes,0=No): "; cin >> tmp; delicate = tmp;
935     cout << "Quantity: "; cin >> qty;
936     cout << "Price per unit ($): "; cin >> price;
937     cout << "Location (e.g. Aisle-1): "; cin >> location;
938
939     InventoryItem* item = nullptr;
940     if (perishable) {
941         cout << "Expiry Date (YYYY-MM-DD): "; cin >> expiry;
942         item = new PerishableItem(name, type, size, colour,
943             hazardous, delicate, qty, location, price, expiry);
944     } else {
945         item = new NonPerishableItem(name, type, size, colour,
946             hazardous, delicate, qty, location, price);
947     }
948
949     // Log through departments
950     receiving.LogDelivery(item->getName(), item->getProductID(), qty);
951     it.log("Received item: " + item->getName() + " (ID:" + item->getProductID() + ")");
952
953     inventory.push_back(item);
954     cout << "[Inventory] Item added! ID: " << item->getProductID()
955         << " | QC Status: Pending" << endl;
956 }
957
958 // 2. View Inventory
959 void viewInventory() const {
960     if (inventory.empty()) { cout << "Inventory is empty." << endl; return; }
```

```

FOREIGN AND LOCALS MANAGEMENT SYSTEM.cpp  programming.cpp X  Extension: Live Share Mob Timer
C:\Users\ADMIN> OneDrive\ Desktop\ PROFRAMMING > 2.2\ Project 2\ programming.cpp > WarehouseSystem > placeCustomerOrder()
810 class WarehouseSystem {
959 void viewInventory() const {
961     for (auto item : inventory) item->displayItem();
962 }
963
964 // 3. QC Inspection
965 void runQCInspection() {
966     qc.inspectItem(inventory);
967     it.log("QC inspection performed.");
968 }
969
970 // 4. Assign Storage Location
971 void updateLocation() {
972     storage.assignLocation(inventory);
973     it.log("Storage location updated.");
974 }
975
976 // 5. Place Customer Order (Order Processing dept validates)
977 void placeCustomerOrder() {
978     if (inventory.empty()) {
979         cout << "No inventory available. Receive items first." << endl; return;
980     }
981     string pID, custName, addr, date;
982     int qty;
983
984     cout << "\n--[Order Processing] Place Customer Order--" << endl;
985     cout << "Product ID: "; cin >> pID;
986     cout << "Quantity: "; cin >> qty;
987     cout << "Customer Name: "; cin >> ws; getline(cin, custName);
988     cout << "Delivery Address: "; getline(cin, addr);
989     cout << "Date (YYYY-MM-DD): "; cin >> date;
990
991     double price = 50.0;
992     for (auto item : inventory)
993         if (item->getProductID() == pID)
994             price = item->getPricePerUnit();
995
996     string oID = "ORD-" + to_string(orderCounter++);
997     auto order = new CustomerOrder(oID, pID, date, qty, custName, addr, price);
998     order->processOrder(inventory); // checks QC status + stock
999
1000     if (order->getStatus() == "Processed") {
1001         finance.recordRevenue(order->getRevenue());
1002         it.log("Customer order placed: " + oID);
1003         procurement.checkReorderLevels(inventory); // auto-flag low stock
1004     }
1005     orders.push_back(order);
1006 }
1007
1008 // 6. Pick & Pack an order

```

C:\Users\ADMIN > OneDrive > Desktop > PROGRAMMING > 2.2 > Project 2 > G- programming.cpp > WarehouseSystem > placeCustomerOrder()

```
1010 class WarehouseSystem {
1008     // 6. Pick & Pack an order
1009     void pickAndPack() {
1010         pickingPacking.packOrder(orders, inventory);
1011         it.log("Picking & packing performed.");
1012     }
1013
1014     // 7. Dispatch / Ship an order
1015     void dispatchShipment() {
1016         shipping.dispatchOrder(orders, inventory);
1017         it.log("Shipment dispatched.");
1018     }
1019
1020     // 8. Place Return Order
1021     void placeReturnOrder() {
1022         if (inventory.empty()) {
1023             cout << "No inventory. Receive items first." << endl; return;
1024         }
1025         string pID, reason, date;
1026         int qty;
1027
1028         cout << "\n--[Returns] Place Return Order--" << endl;
1029         cout << "Product ID to return: "; cin >> pID;
1030         cout << "Quantity: "; cin >> qty;
1031         cout << "Return Reason: "; cin >> ws; getline(cin, reason);
1032         cout << "Date (YYYY-MM-DD): "; cin >> date;
1033
1034         string rID = "RET-" + to_string(returnCounter++);
1035         auto ret = new ReturnOrder(rID, pID, date, qty, reason);
1036         orders.push_back(ret);
1037         cout << "[Returns] Return order " << rID
1038             << " logged. Use Department Operations -> Returns to approve/reject." << endl;
1039         it.log("Return order logged: " + rID);
1040     }
1041
1042     // 9. View all orders
1043     void viewAllOrders() const {
1044         if (orders.empty()) { cout << "No orders placed yet." << endl; return; }
1045         cout << "\n--- All Orders ---" << endl;
1046         for (auto o : orders) o->displayOrder();
1047     }
1048
1049     // 10. Finance Report
1050     void financeReport() {
1051         finance.showFinanceReport(people, allDepts);
1052     }
1053
1054     // -----
1055     // DEPARTMENT OPERATIONS SUB-MENU
1056     // -----
1057     void departmentMenu() {
```

C:\Users\ADMIN> OneDrive\ Desktop> PROGRAMMING> 2.2> Project 2> G:\programming.cpp> WarehouseSystem> placeCustomerOrder()

```
810 class WarehouseSystem {
1054 // -----
1056 // -----
1057 void departmentMenu() {
1058     int choice;
1059     do {
1060         cout << "\n==== Department Operations =====< endl;
1061         cout << " 1. Receiving      - view delivery log"    << endl;
1062         cout << " 2. Quality Control - inspect an item"      << endl;
1063         cout << " 3. Storage        - update item location"   << endl;
1064         cout << " 4. Order Processing - view order status"    << endl;
1065         cout << " 5. Picking & Packing - pack an order"       << endl;
1066         cout << " 6. Shipping        - dispatch an order"    << endl;
1067         cout << " 7. Procurement    - check reorder levels"  << endl;
1068         cout << " 8. Returns        - approve/reject a return" << endl;
1069         cout << " 9. HR            - staff & payroll"         << endl;
1070         cout << "10. Finance        - revenue & profit report" << endl;
1071         cout << "11. IT/Systems    - log & system info"       << endl;
1072         cout << "12. All Dept Summary" << endl;
1073         cout << " 0. Back to main menu" << endl;
1074         cout << "Choice: ";
1075         cin >> choice;
1076
1077         switch (choice) {
1078             case 1: receiving.showDeliveryLog();             break;
1079             case 2: runQCInspection();                       break;
1080             case 3: updateLocation();                        break;
1081             case 4: orderProcessing.showOrderStatus(orders); break;
1082             case 5: pickAndPack();                          break;
1083             case 6: dispatchShipment();                     break;
1084             case 7: procurement.checkReorderLevels(inventory); break;
1085                 |   procurement.showPurchaseRequests();    break;
1086             case 8: returnsDept.processReturn(orders, inventory); break;
1087             case 9: hr.showAllStaff(people);                break;
1088                 |   hr.showPayroll(people);                 break;
1089             case 10: financeReport();                       break;
1090             case 11: it.showLog();                          break;
1091                 |   it.showSystemInfo();                    break;
1092             case 12:
1093                 cout << "\n--- All Department Summaries ---< endl;
1094                 for (auto d : allDepts) d->displaySummary();
1095                 break;
1096             case 0: break;
1097             default: cout << "Invalid choice." << endl;
1098         }
1099     } while (choice != 0);
1100 }
1101
1102 public:
1103     WarehouseSystem() {
1104         // Register non-owning pointers to all depts for summary loops
```

C:\Users\ADMIN > OneDrive > Desktop > PROGRAMMING > 2.2 > Project 2 > C:\programming.cpp > WarehouseSystem > placeCustomerOrder()

```
810 class WarehouseSystem {
1102 public:
1103     WarehouseSystem() {
1104         // Register non-owning pointers to all depts for summary loops
1105         allDepts = { &receiving, &qc, &storage, &orderProcessing,
1106                     &pickingPacking, &shipping, &procurement,
1107                     &returnsDept, &hr, &finance, &it };
1108     }
1109     runCheckpoints();
1110     loadDummyData();
1111 }
1112
1113 ~WarehouseSystem() {
1114     for (auto i : inventory) delete i;
1115     for (auto p : people) delete p;
1116     for (auto o : orders) delete o;
1117     // allDepts are non-owning (point to member objects) - do NOT delete
1118 }
1119
1120 void run() {
1121     int choice;
1122     do {
1123         cout << "\n==== Foreigns & Locals WMS =====< endl;
1124         cout << "1. Receive New Item (Receiving -> Inventory)" << endl;
1125         cout << "2. View Inventory" << endl;
1126         cout << "3. Place Customer Order" << endl;
1127         cout << "4. Place Return Order" << endl;
1128         cout << "5. View All Orders" << endl;
1129         cout << "6. Finance & Profit Report" << endl;
1130         cout << "7. Department Operations" << endl;
1131         cout << "8. Exit" << endl;
1132         cout << "Enter choice: ";
1133         cin >> choice;
1134
1135         switch (choice) {
1136             case 1: receiveItem(); break;
1137             case 2: viewInventory(); break;
1138             case 3: placeCustomerOrder(); break;
1139             case 4: placeReturnOrder(); break;
1140             case 5: viewAllOrders(); break;
1141             case 6: financeReport(); break;
1142             case 7: departmentMenu(); break;
1143             case 8: cout << "Exiting. Goodbye!" << endl; break;
1144             default: cout << "Invalid choice. Please try again." << endl;
1145         }
1146     } while (choice != 8);
1147 }
1148 };
1149
1150 // =====
1151 // MAIN
```

C:\Users\ADMIN > OneDrive > Desktop > PROGRAMMING > 2.2 > Project 2 > G- programming.cpp > WarehouseSystem > placeCustomerOrder()

```
810 class WarehouseSystem {
1120 void run() {
1122     do {
1125         cout << "2. View Inventory" << endl;
1126         cout << "3. Place Customer Order" << endl;
1127         cout << "4. Place Return Order" << endl;
1128         cout << "5. View All Orders" << endl;
1129         cout << "6. Finance & Profit Report" << endl;
1130         cout << "7. Department Operations" << endl;
1131         cout << "8. Exit" << endl;
1132         cout << "Enter choice: ";
1133         cin >> choice;
1134
1135         switch (choice) {
1136             case 1: receiveItem(); break;
1137             case 2: viewInventory(); break;
1138             case 3: placeCustomerOrder(); break;
1139             case 4: placeReturnOrder(); break;
1140             case 5: viewAllOrders(); break;
1141             case 6: financeReport(); break;
1142             case 7: departmentMenu(); break;
1143             case 8: cout << "Exiting. Goodbye!" << endl; break;
1144             default: cout << "Invalid choice. Please try again." << endl;
1145         }
1146     } while (choice != 8);
1147 }
1148 };
1149
1150 // =====
1151 // MAIN
1152 // =====
1153 int main() {
1154     WarehouseSystem wms;
1155     wms.run();
1156     return 0;
1157 }
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS COMMENTS

```
-----
[Customer Order]
Order ID:   ORD-001
Date:      2026-05-01
Product ID: FOOD-M-WHT
Quantity:   5
Customer:  Manjiku Njoroge
Deliver to: Githurai 45
Total Cost: $250.00
Status:    Processed
-----

[Return Order]
Order ID:   RET-001
Date:      2026-05-02
Product ID: FOOD-M-WHT
Quantity:   2
Return Reason: Damaged on arrival
Approval Status: Approved
Order Status: Processed
-----

===== Foreigns & Locals WMS =====
1. Receive New Item (Receiving r&f Inventory)
2. View Inventory
3. Place Customer Order
4. Place Return Order
5. View All Orders
6. Finance & Profit Report
7. Department Operations
8. Exit
Enter choice:
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS COMMENTS

Code + - [] [] ... [] >

Product ID: FOOD-M-WHT
Quantity: 2
Return Reason: Damaged on arrival
Approval Status: Approved
Order Status: Processed

===== Foreigns & Locals WMS =====

1. Receive New Item (Receiving & Inventory)
 2. View Inventory
 3. Place Customer Order
 4. Place Return Order
 5. View All Orders
 6. Finance & Profit Report
 7. Department Operations
 8. Exit
- Enter choice: 1

--[Receiving Dept] Enter Delivery Details--

Name: HP-Pavilion
Type (FOOD/ELEC/CHEM): ELEC
Size (S/M/L): L
Colour: grey
Hazardous? (1=Yes,0=No): 1
Perishable? (1=Yes,0=No): 0
Fragile? (1=Yes,0=No): 1
Quantity: 2
Price per unit (\$): 600
Location (e.g. Aisle-1): Aisle-1
[Receiving] Delivery received & logged. Sending to Quality Control.
[Inventory] Item added! ID: ELEC-L-gre-H-D | QC Status: Pending

===== Foreigns & Locals WMS =====

1. Receive New Item (Receiving & Inventory)
 2. View Inventory
 3. Place Customer Order
 4. Place Return Order
 5. View All Orders
 6. Finance & Profit Report
 7. Department Operations
 8. Exit
- Enter choice:

```
--[Receiving Dept] Enter Delivery Details--
Name: HP-Pavilion
Type (FOOD/ELEC/CHEM): ELEC
Size (S/M/L): L
Colour: grey
Hazardous? (1=Yes,0=No): 1
Perishable? (1=Yes,0=No): 0
Fragile? (1=Yes,0=No): 1
Quantity: 2
Price per unit ($): 600
Location (e.g. Aisle-1): Aisle-1
[Receiving] Delivery received & logged. Sending to Quality Control.
[Inventory] Item added! ID: ELEC-L-gre-H-D | QC Status: Pending
```

===== Foreigns & Locals WMS =====

1. Receive New Item (Receiving Γ&A Inventory)
 2. View Inventory
 3. Place Customer Order
 4. Place Return Order
 5. View All Orders
 6. Finance & Profit Report
 7. Department Operations
 8. Exit
- Enter choice: 7

===== Department Operations =====

- | | |
|----------------------|-----------------------------|
| 1. Receiving | ΓÇö view delivery log |
| 2. Quality Control | ΓÇö inspect an item |
| 3. Storage | ΓÇö update item location |
| 4. Order Processing | ΓÇö view order status |
| 5. Picking & Packing | ΓÇö pack an order |
| 6. Shipping | ΓÇö dispatch an order |
| 7. Procurement | ΓÇö check reorder levels |
| 8. Returns | ΓÇö approve/reject a return |
| 9. HR | ΓÇö staff & payroll |
| 10. Finance | ΓÇö revenue & profit report |
| 11. IT/Systems | ΓÇö log & system info |
| 12. All Dept Summary | |
| 0. Back to main menu | |

Choice: █